컴파일러의 기초 project 3 보고서

전기정보공학부 2021-15905 홍성준

November 21, 2025

1 Build

project3에서 추가된 소스 파일은 `decl.c`와 `error.c` 총 두 개이다. `decl.c`은 push_scope, declare, make_var와 같이 decl 구조체를 만들거나 scope에 접근하는 함수들로 구성되어 있다. `error.c`는 check_redeclaration, check_binary처럼 decl 구조체를 받아 semantically valid한지 검증하고, 에러 메시지를 내는 함수들로 구성되어 있다.

하나의 line에 여러 error가 존재할 때를 커버하기 위해 `pass_tdecl` 및 `errflag` global variable을 사용하였다. `pass_tdecl`은 에러가 발생했을 때, reduce되는 nonterminal에게 전달하는 decl 구조체이고, `errflag`는 에러가 발생했을 때 1로 초기화되어 같은 라인에서 또다른 에러가 발생했을 때 에러 메시지 출력을 막도록 하였다. `errflag`는 `subc.y`에서 '\n'을 만나면 0으로 다시 꺼지도록 하였다.

2 code

2.1 ext_def

global variable 및 function, struct를 선언하는 부분인 `ext_def`를 아래 Figure 1와 같이 구현하였다.

```
ext_def
  : type_specifier ID ';' {
    if (!check_redeclaration($2, TRUE)) declare($2, make_var($1));
  }
  | type_specifier ID '[' INTEGER_CONST ']' ';' {
    if (!check_redeclaration($2, TRUE)) declare($2, make_const(make_arr($4, $1)));
  }
  | struct_specifier ';'     {}
  | func_decl {
    push_scope();
    insert_ste_list($1->formals);
  } compound_func_stmt {
    pop_scope(0);
  }
  ;
```

Figure 1: `ext_def`

declare 함수를 통해 id와 decl 구조체를 묶어 scope에 등록해주었고, function declaration의 경우엔 compound_func_stmt에 들어가기 전 scope를 push하고, 해당 블럭을 벗어난 시점에서 pop_scope를 하도록 action rule을 구성하였다. function의 경우 statement block에 들어가기 전, parameter들을 scope에 등록해줄 필요가 있기에 아래 Figure 2처럼 compound_stmt, compound_func_stmt로 구분되도록 기존의 grammar를 수정하였다.

```
compound_stmt
  : '{' { push_scope(); } def_list stmt_list '}' { pop_scope(0); }
  ;

compound_func_stmt
  : '{' def_list stmt_list '}'
  ;
```

Figure 2: `compound_stmt`

2.2 type_specifier, struct_specifier

type을 정의해주는 부분인 type_specifier와 struct_specifier 부분을 아래 Figure 3와 같이 구현하였다.

```
type_specifier
  : TYPE                  { $$ = $1; }
  | TYPE '*'              { $$ = make_ptr($1); }
  | struct_specifier      { $$ = $1; }
  | struct_specifier '*'  { $$ = make_ptr($1); }
  ;

struct_specifier
  : STRUCT ID {
    if (!check_redeclaration($2, TRUE)) {
      $<declptr>$ = make_str(NULL);
      declare_glob($2, $<declptr>$);
    }
    else $<declptr>$ = pass_tdecl;
  } '{' {
    push_scope();
  } def_list '}' {
    if ($<declptr>3 == pass_tdecl) pop_scope(0);
    else $<declptr>3->fields = pop_scope(0);
    $$ = $<declptr>3;
  }
  | STRUCT ID { $$ = (!check_incomplete($2))? lookup($2) : pass_tdecl; }
  ;
```

Figure 3: `type_specifier, struct_specifier`

struct를 정의할 땐 redelaration check를 진행한 후, declare_glob를 이용해 global type으로 scope에 저장해 주었고, struct를 사용할 땐, incomplete check를 먼저 진행하여 정의돼있는 struct인지 확인한 후 scope를 lookup하여 specifier에 전달해주도록 하였다.

2.3 func_decl, param_decl, args

function declaration 및 parameter, argument와 관련된 부분을 아래 Figure 4, 5, 6와 같이 구현하였다. function declaration의 경우, return statement에서 return type checking을 진행할 수 있도록 하기 위해 함수의 return type을 returnid로 scope에 push한 후에 parameter를 push하도록 하였다. function declaration이나 parameter declaration 모두 declare 시 redeclaration check를 진행했고, argument를 연결할 때 동일한 type decl을 사용하는 문제를 해결하기 위해 make_arg 함수 내에서 var decl을 만든 후 next field를 사용해 연결하도록 하였다.

```
func_decl
  : type_specifier ID '(' {
    $<declptr>$ = make_func($1);
    if (!check_redeclaration($2, TRUE)) declare($2, $<declptr>$);
    push_scope();
    declare(returnid, make_var($1));
  } ')' {
    $<declptr>4->formals = pop_scope(0);
    $$ = $<declptr>4;
  }
  | type_specifier ID '(' {
    $<declptr>$ = make_func($1);
    if (!check_redeclaration($2, TRUE)) declare($2, $<declptr>$);
    push_scope();
    declare(returnid, make_var($1));
  } param_list ')' {
    $<declptr>4->formals = pop_scope(0);
    $$ = $<declptr>4;
  }
  ;
```

Figure 4: `func_decl`

```
param_list
  : param_decl                {}
  | param_list ',' param_decl   {}
  ;

param_decl
  : type_specifier ID {
    if (!check_redeclaration($2, FALSE)) declare($2, make_var($1)); }
  | type_specifier ID '[' INTEGER_CONST ']' {
    if (!check_redeclaration($2, FALSE)) declare($2, make_const(make_arr($4, $1)));
  }
  ;
```

Figure 5: `param_decl`

```
args
  : expr          { $$ = make_var($1); }
  | args ',' expr { $$ = make_arg($3, $1); }
  ;
```

Figure 6: `args`

2.4 def

local variable의 declaration 부분인 `def`를 아래 Figure 7와 같이 구현하였다. 마찬가지로 redeclaration check 후 declare 함수를 통해 ste scope에 등록해주었다.

```
def_list
  : def_list def  {}
  | %empty        {}
  ;

def
  : type_specifier ID ';' {
    if (!check_redeclaration($2, FALSE)) declare($2, make_var($1));
  }
  | type_specifier ID '[' INTEGER_CONST ']' ';' {
    if (!check_redeclaration($2, FALSE)) declare($2, make_const(make_arr($4, $1)));
  }
  ;
```

Figure 7: `def`

2.5 expr

assignment를 수행하거나 binary symbol이 reduce되는 `expr`을 아래 Figure 8와 같이 구현하였다. assignment에 대한 error check는 LHS가 lvalue인지, RHS가 NULL일 때 LHS가 포인터 타입인지, LHS와 RHS의 타입이 같은지를 차례대로 확인하도록 하였다. 또한, assignment 결과는 const이어야 하므로 int_tdecl이나 char_tdecl일 경우 int_tdecl_const나 char_tdecl_const를 전달해주었다. int_tdecl과 int_tdecl_const 모두 type decl 구조체이나, $unary \rightarrow (expr)$ production에 의해 reduce될 때 expr의 type decl에 해당 type decl이 const로부터 왔는지, 혹은 var로부터 온건지, const 정보를 담고 있게 하기 위해 구분하였다. suffix로 const가 붙은 global type decl 구조체는 isconst field가 1로서, $(expr) = expr$이나 $\&(expr)$ 형태의 sentential form을 만났을 때, isconst field 값을 확인해 assignable check와 addressof check를 수행할 수 있도록 하였다.

```
expr
  : unary '=' expr {
    if (check_assignable($1) || check_null($1, $3) || check_incompatible($1, $3)) $$ = pass_tdecl;
    else {
      if ($3 == int_tdecl) $$ = int_tdecl_const;
      else if ($3 == char_tdecl) $$ = char_tdecl_const;
      else $$ = $3;
    }
  }
  | binary { $$ = $1; }
  ;
```

Figure 8: `expr`

2.6 binary

두 operand에 대한 연산을 수행하는 부분인 `binary`를 아래 Figure 9와 같이 구현하였다. 두 operand의 타입이 같은지, operator에 valid한 operand인지를 확인한 후, const type decl인 int_tdecl_const를 전달해주도록 하였다.

```
binary
  : binary RELOP binary {
    if (check_comparable($1, $3, (TYPE_INT | TYPE_CHAR))) $$ = pass_tdecl;
    else $$ = int_tdecl_const;
  }
  | binary EQUOP binary {
    if (check_comparable($1, $3, (TYPE_INT | TYPE_CHAR | TYPE_PTR))) $$ = pass_tdecl;
    else $$ = int_tdecl_const;
  }
  | binary '+' binary {
    if (check_binary($1, $3, TYPE_INT)) $$ = pass_tdecl;
    else $$ = int_tdecl_const;
  }
  | binary '-' binary {
    if (check_binary($1, $3, TYPE_INT)) $$ = pass_tdecl;
    else $$ = int_tdecl_const;
  }
  | binary '*' binary {
    if (check_binary($1, $3, TYPE_INT)) $$ = pass_tdecl;
    else $$ = int_tdecl_const;
  }
  | binary '/' binary {
    if (check_binary($1, $3, TYPE_INT)) $$ = pass_tdecl;
    else $$ = int_tdecl_const;
  }
  | binary '%' binary {
    if (check_binary($1, $3, TYPE_INT)) $$ = pass_tdecl;
    else $$ = int_tdecl_const;
  }
  | unary %prec '=' { $$ = $1->type; }
  | binary LOGICAL_AND binary {
    if (check_binary($1, $3, TYPE_INT)) $$ = pass_tdecl;
    else $$ = int_tdecl_const;
  }
  | binary LOGICAL_OR binary {
    if (check_binary($1, $3, TYPE_INT)) $$ = pass_tdecl;
    else $$ = int_tdecl_const;
  }
  ;
```

Figure 9: `binary`

2.7 unary

`unary` nonterminal에 대한 production은 아래 Figure 10와 같이 구현하였다. $unary \rightarrow (expr)$의 경우, expr의 isconst field 값을 보고, const에서 온 type decl이면 const decl을 생성하고, 아니면 var decl을 생성하도록 하였다. $unary \rightarrow ID$의 경우, undeclared check 수행 후 ste scope lookup 결과를 전달해 주었고, array, function call, struct의 경우엔 각각의 access 함수를 사용해 관련 error check를 진행한 후 var이나 const decl를 전달하도록 하였다.

```
unary
  : '(' expr ')'            { $$ = ($2->isconst)? make_const($2) : make_var($2); }
  | INTEGER_CONST           { $$ = make_const(int_tdecl_const); $$->intval = $1; }
  | CHAR_CONST              { $$ = make_const(char_tdecl_const); $$->charval = $1; }
  | STRING                  { $$ = make_const(string_tdecl); $$->stringval = $1; }
  | ID                      { $$ = (!check_undeclared($1))? lookup($1) : make_const(pass_tdecl); }
  | '-' unary %prec '!'     { $$ = (!check_unary($2, TYPE_INT))? make_const($2->type) : make_const(pass_tdecl); }
  | '!' unary               { $$ = (!check_unary($2, TYPE_INT))? make_const($2->type) : make_const(pass_tdecl); }
  | unary INCOP %prec '.' { $$ = (!check_unary($1, TYPE_INT | TYPE_CHAR))? make_const($1->type) : make_const(pass_tdecl); }
  | unary DECOP %prec '.' { $$ = (!check_unary($1, TYPE_INT | TYPE_CHAR))? make_const($1->type) : make_const(pass_tdecl); }
  | INCOP unary             { $$ = (!check_unary($2, TYPE_INT | TYPE_CHAR))? make_const($2->type) : make_const(pass_tdecl); }
  | DECOP unary             { $$ = (!check_unary($2, TYPE_INT | TYPE_CHAR))? make_const($2->type) : make_const(pass_tdecl); }
  | '&' unary               { $$ = (!check_addressof($2))? make_const(make_ptr($2->type)) : make_const(pass_tdecl); }
  | '*' unary %prec '!'     { $$ = (!check_indirection($2))? make_var($2->type->ptrto) : make_var(pass_tdecl); }
  | unary '[' expr ']'      { $$ = access_arr($1, $3); }
  | unary '.' ID            { $$ = access_struct($1, $3); }
  | unary STRUCTOP ID       { $$ = access_structp($1, $3); }
  | unary '(' args ')'      { $$ = access_function($1, $3); }
  | unary '(' ')'           { $$ = access_function($1, NULL); }
  | SYM_NULL                { $$ = make_const(make_ptr(null_tdecl)); }
  ;
```

Figure 10: `unary`

3 Result

작성한 code를 build하여 주어진 open_test에 대해 semantic analyze를 진행한 결과는 아래 Figure 11와 같았다. 적절한 에러를 라인당 하나씩 잘 출력하는 것을 확인할 수 있었다.

```
Testcase: testcase/str_op.c
testcase/str_op.c:17: error: redeclaration
testcase/str_op.c:35: error: incompatible types for assignment operation
testcase/str_op.c:39: error: incompatible types for assignment operation
testcase/str_op.c:41: error: incompatible types for assignment operation
testcase/str_op.c:44: error: incompatible types for assignment operation

Testcase: testcase/var_def.c
testcase/var_def.c:7: error: redeclaration
testcase/var_def.c:9: error: redeclaration
testcase/var_def.c:14: error: use of undeclared identifier
```

```
Testcase: testcase/func_op.c
testcase/func_op.c:20: error: incompatible return types
testcase/func_op.c:34: error: incompatible arguments in function call
testcase/func_op.c:35: error: incompatible arguments in function call
testcase/func_op.c:39: error: incompatible arguments in function call
testcase/func_op.c:41: error: incompatible types for assignment operation
testcase/func_op.c:42: error: incompatible arguments in function call

Testcase: testcase/ptr_op.c
testcase/ptr_op.c:9: error: lvalue is not assignable
testcase/ptr_op.c:13: error: cannot take the address of an rvalue
testcase/ptr_op.c:15: error: incompatible types for assignment operation
testcase/ptr_op.c:16: error: invalid operands to binary expression
testcase/ptr_op.c:17: error: invalid argument type to unary expression
testcase/ptr_op.c:18: error: invalid argument type to unary expression
testcase/ptr_op.c:20: error: incompatible types for assignment operation
testcase/ptr_op.c:21: error: incompatible types for assignment operation
testcase/ptr_op.c:23: error: incompatible types for assignment operation
testcase/ptr_op.c:24: error: invalid argument type to unary expression
testcase/ptr_op.c:25: error: invalid argument type to unary expression
testcase/ptr_op.c:27: error: types are not comparable in binary expression
```

Figure 11: Result of open_test